



version_1

v1 Scenario

🎯 v1 목표

고객이 카카오톡 메시지 입력 → Orchestrator → Intake Agent → Booking Agent → Notification Agent
→ 고객 응답 흐름 고정

예약 문의 → 정보 수집 → 예약 가능 여부 판단 → 예약 등록 → 선입금 대기/확정
고객 메시지 수신

- Intake
- slot 부족하면 추가질문
- slot 충분하면 Booking/Policy 검증
- 가능하면 provisional reservation 생성
- deposit required 안내
- 입금 완료 이벤트 오면 confirmed 처리

- 구현 사항

2개의 핵심 agent + 1개의 coordinator + 1개의 response/message layer

- **Workflow orchestrator** → agent라기보단 **state router / coordinator**
 - **Intake Agent** → 진짜 LLM agent
 - **Booking/Policy Agent** → LLM보다 규칙 기반 판정 노드 + backend tool 호출
 - **Notification Agent** → **Response Generator + Notification Service**로 나누기
-
- kakao json을 db에 넣을 json으로 변환 후 백엔드에 넘겨주기 ⇒ AI
 - kakao agent 수신 = agent 담당

2. 필요한 Agent 종류

1. Workflow orchestrator (main 컨트롤 관리자)

- ★ 역할: 현재 상태를 보고 다음 단계를 결정

- 어느 모듈을 호출할 지 정함

고객으로부터 채팅창에 새 문의 들어옴 → 가장 먼저 intake agent 호출
 손님이 예약 관련 문의를 하면 → Booking 검증 agent 호출
 선입금 필요하면 → payment 대기 상태로 전이
 입금 완료되면 → confirmation 처리

- 현재 state 보고 다음 node 결정
- missing field 있으면 follow-up
- 예약 등록에 필요한 slot들 전부 채웠다면 booking validation
 - ver1은 일단 여기서 booking 검증 다 끝나면 바로 예약 정보 DB에 저장하고 끝
- 확장판인 ver2의 경우(나중의 작업이지만 미리 정리):
 - validation 통과하면 provisional booking 생성
 - deposit pending이면 해당 상태 유지
 - payment event 오면 confirmation 처리

2. Intake Agent

- Input: 고객의 unstructured format의 natural language
 - 예시: "예약 문의하고 싶어요!" / "안녕하세요, 예약 취소하고자 연락드립니다." / "예약 문의"(오타) / 등등
 - ★ 역할1: 입력된 고객의 unstructured format의 자연어 발화를 형식에 잘 맞도록 정돈하고 다듬는 역할
 - ★ 역할2: 잘 정돈된 자연어 발화 → 의도(intent) 분류 → 사용할 slot 추출
 - **intent classification**
 - 의도 (=status) (각 종류별 변수명)
 - `booking` : 예약 등록(신규/기존 고객 여부인지와는 무관하게)
 - `question` : 기타 문의사항
 - `change_booking` : 예약 변경
 - `cancel_booking` : 예약 취소
 - 일단 ver1은 status == `booking` 만 먼저 구현해보는 것을 목표로 한다.
- ⇒ 즉, intake agent는 **의도 분류 및 자연어 발화 교정** (ex. 오늘, 내일, 이번주...) 기능이 핵심!
- 아마도 마치 챗봇과 같은 agent?
- Output format 예시 (intent == booking인 경우의 output)
 - 아직 booking validation은 하기 전이고, 오직 사용자의 자연어 발화의 intent를 파악하여 예약 등록 agent로 넘겨주기 위한 자연어를 만드는 단계이므로 아래와 같이 필수 입력 사항인 name이 missing

field로서 값이 비어있을 수도 있음. 이런 상황도 고려해야 함.

- *성함:
- *전화번호 (010-0000-0000): 010-1234-5678
- *젤제거 유무(O/X): O
- *예약 희망 날짜 (형식: 2026-04-12): 2026-05-04
- *예약 희망 시간 (형식: 18:00): 17:00
- *원하시는 시술 종류(손톱 케어/기본네일/젤네일/페디큐어 등): 젤네일
- *과거 방문경험(O/X): X
- 알게된 경로(간판, 검색, 네이버 블로그, 인스타그램, 지인 소개 등): 인터넷 검색
- 기타 요청 사항(각종 요청/커스텀 디자인/등등):

3. Booking agent

- 역할: 예약 가능 여부 확인 & 영업 시간 검사 & 시술 소요 시간 계산 & 다른 기존 예약들과의 겹침 검사
→ 여기서 걸리는 경우: 예약 등록하지 않고 **대체 가능 시간** 추천
→ 여기서 통과되어 예약 가능할 경우: 선입금 필요 여부 확인 → 예약 등록 (end)

예약 요청을 먼저 받고, 그 시간에 예약이 가능하면 먼저 그걸 캘린더에 등록을 해놓고 선입금을 그 후에 기다리는 방식 → 선입금이 일정시간동안 안되었다면 예약 취소도 해야하는데 이 기능은 version1에서는 일단 생략.

- Input:
 - Intake agent가 제공해준 output(고객의 자연어 응답 정돈한 것)
 - 예약 DB (백엔드)
- 이후 booking validation과 같은 다음 단계 작업을 위해 자연어 응답(input) → **structured JSON format**으로 가공하는 중간과정 필요

```
{
  "intent": "booking",
  "slots": {
    "name" : "",
    "phone_num" : "010-1234-5678",
    "off_removal": true,
    "reserve_date": "2026-05-04",
    "reserve_time": "17:00",
    "service_code": "GEL_BASIC",
    "past_visit": false,
  },
  "missing_fields": ["name"],
  "uncertain_fields": [],
  "need_followup": true,
  "followup_question": "예약자 성함도 알려주실 수 있을까요?",
}
```

```
"decision": "ask_followup"
}
```

- 위와 같이 가공한 JSON을 토대로 이제 policy engine과 대조하면서 마지막으로 이 예약을 승인할 수 있는지 없는지 파악하기

- **Booking agent** Output format(아직 확정 못함)

```
{
  "bookable": false,
  "reason": "time_conflict", // false인 경우 그 이유
  "alternatives": [ // 해당 날짜에 대체 가능한 다른 시간 추천
    "2026-04-12T19:00:00",
    "2026-04-13T11:00:00"
  ],
  "deposit_required": true,
  "estimated_duration_min": 90,
  "booking_status": "not_bookable",
  "next_action": "suggest_alternatives"
}
```

- 참고로 이 agent에서 모든 판단의 근거는 LLM의 prompt가 아니라, 우리가 정의해놓은 policy data와 코드 여야 함!

4. Policy engine

- 메인 policy는 대부분 신규예약과 관련되게 맞지만, policy에 신규예약과 관련되지 않은 별개의 정책이 있을 수 있고, booking agent 말고도 다른 agent에서도 policy 대조를 할 수 있으므로 따로 policy Engine을 분리하자!!
- 여기서 policy란: 여러 agent들마다 수행하는 예약 및 각종 작업들에 필요한 네일샵 가게의 정책 policy를 뜻함

신규 예약 관련(booking):

- 영업시간(10:00-22:00) 안에만 예약 가능
- 월요일은 정기휴무날
- 각 시술별 소요 시간: 기본 네일은 30분, 젤 네일은 1시간, 젤제거 안했을 경우 젤 제거 작업: 30분
- 기본 예약금: 2만원
- 예약 생성 후 30분 안에 결제 안 하면 자동 취소

변경(change_booking):

- 예약 하루 전까지만 변경 가능
- 당일 변경은 사장님 승인 필요 -> human-in-the-loop

취소(cancel_booking):

- 하루 전 취소는 환불 + 수수료 1만원
- 당일 취소는 환불 불가

결제(payment_pending):

- 예약 등록 후, 선입금 기한 30분 초과 시 예약 취소
- 부분 입금 역시 예약 등록 불가 (취소)

노쇼:

- 노쇼 횟수 n회 이상이면 선결제 필수
- 노쇼 발생 시 다음 예약 제한

[ver1 시나리오]

- 처음 뵈는 인삿말로 `Orchestrator` 가 먼저 아래와 같이 intent classification을 유도하는 고정된 인삿말 메시지를 먼저 채팅창에 출력

안녕하세요! 숙명네일샵입니다^^
예약문의를 원하시면 '예약 문의'
예약변경을 원하시면 '예약 변경'
예약취소를 원하시면 '예약 취소'
그 외 각종 문의는 '기타'
를 입력해주세요!

- 채팅창에 들어온 사용자는 일반적으로 위 인삿말에서 명시하는 intent 중 하나가 담긴 자연어 발화를 입력
 - ex) "예약 문의" / "예약 하고 싶어요!!" / "혹시 저 예약 변경할 수 있나요?" / "기타" / "회원권 얼마예요?" / 등등 다양한 unstructured format의 자연어 발화로 들어옴
- 고객이 위 인삿말 메시지 직후 어떤 자연어 발화를 넣으면, 일단 바로 `intake agent` 를 호출
 - 고객의 자연어 입력을 먼저 분석하고 잘 다듬은 후(오타 교정, 문장 정리 등) → llm (openai API)으로 문맥 파악하여 의도 분류
 - ver1에서는 일단 의도들 중 신규예약 등록 기능만을 다루기로 했으니까, 일단 `intent == booking` 분류만 구현해볼 예정
- 만약 `intent == booking` 으로 분류된 경우
 - intake agent가 아래의 예약 형식 format을 고정값으로 뱉는다
 - ▼ 고정값은 db한테 받아오기 (사전에 db에 저장되어 있음)

안녕하세요 고객님~ 예약 문의 주셔서 감사합니다:)
아래 예약 형식에 맞게 채워서 보내주시면 확인 후 예약 도와드리겠습니다!! (* 표시는 필수사항)
- *성함:
- *전화번호 (010-0000-0000):
- *젤제거 유무(O/X):
- *예약 희망 날짜 (형식: 2026-04-12):
- *예약 희망 시간 (형식: 18:00):
- *원하시는 시술 종류(손톱 케어/기본네일/젤네일/페디큐어 등):
- *과거 방문경험(O/X):
- 알게된 경로(간판, 검색, 네이버 블로그, 인스타그램, 지인 소개 등):
- 기타 요청 사항(각종 요청/커스텀 디자인/등등):

- 고객이 위 형식에 각 slot에 해당하는 부분에 아래와 같이 정보를 채운 자연어 문자열을 사장님에게 전송

- *성함: 김지수
- *전화번호 (010-0000-0000): 010-1234-5678
- *젤제거 유무(O/X): O
- *예약 희망 날짜 (형식: 2026-04-12): 2026-05-04
- *예약 희망 시간 (형식: 18:00): 17:00
- *원하시는 시술 종류(손톱 케어/기본네일/젤네일/페디큐어 등): 젤네일
- *과거 방문경험(O/X): X
- 알게된 경로(간판, 검색, 네이버 블로그, 인스타그램, 지인 소개 등): 인터넷 검색
- 기타 요청 사항(각종 요청/커스텀 디자인/등등):

- 고객으로부터 위 문자열을 받으면, `intake agent` 가 이 문자열을 `booking agent` 에게 전달
 - 이제 실제로 예약을 "등록"하는 단계부터는 `booking agent` 의 담당!
- 그 후 `booking agent` 의 작업 순서:
 - 1) 예약 정보 문자열에서 **slot extraction**
 - 2) **missing slot detection**(= 필수 정보가 부족한 경우 예약 DB에 등록을 보류하고 고객에게 정보 추가 요청)
 - 3) 마지막으로 네일샵 기본 policy에 어긋나지 않는지 검사 → 별도로 정의해놓은 `policy engine` 을 사용해서 네일샵의 policy와 사용자 예약 정보 비교하기
(e.g., 고객이 예약을 원하는 시간대인 토요일 오후 3시가 영업시간인지, 해당 시간대에 젤네일(1시간 소요) 시술 시간이 가능한지, 휴무일은 아닌지 등등 예약 가능 여부를 파악)
 - 4) 다 통과했다면 최종적으로 예약이 가능한 것이므로 예약 정보 DB에 등록!

(policy engine 속 네일샵 policy 예시)

신규 예약 관련(booking):

- 영업시간(10:00-22:00) 안에만 예약 가능
- 월요일은 정기휴무날
- 각 시술별 소요 시간: 기본 네일은 30분, 젤 네일은 1시간, 젤제거 안했을 경우 젤 제거 작업: 30분
- 기본 예약금: 2만원
- 예약 생성 후 30분 안에 결제 안 하면 자동 취소

변경(change_booking):

- 예약 하루 전까지만 변경 가능
- 당일 변경은 사장님 승인 필요 -> human-in-the-loop

취소(cancel_booking):

- 하루 전 취소는 환불 + 수수료 1만원
- 당일 취소는 환불 불가

결제(payment_pending):

- 예약 등록 후, 선입금 기한 30분 초과 시 예약 취소
- 부분 입금 역시 예약 등록 불가 (취소)

노쇼:

- 노쇼 횟수 n 회 이상이면 선결제 필수
- 노쇼 발생 시 다음 예약 제한

- a. 필요한 정보가 다 들어왔고, 각 slot에 들어갈 정보들이 다 잘 정제까지 가능한 형태이고, 네일샵의 기본 policy를 다 충족하여 예약 등록까지 가능한 상태라면 그대로 예약 정보 DB에 저장하고 끝.
- b. 만약 필요한 정보가 하나라도 빠먹은채(비어있는채로) 전송했다면, 해당 slot명을 다시 고객에게 요청하도록 함 (e.g., 성함을 빠먹은 경우, '성함 정보도 채워서 다시 보내주시겠어요?' 라는 멘트를 고객에게 전송하기)
 - 그 후 missing slot까지 다시 채워서 잘 보내주면 예약 정보는 다 모은 것이므로, 마지막으로 네일샵의 policy와 시간, 날짜 slot을 확인하고 잘 대조하여 최종적으로 해당 날짜 시간대에 예약 등록이 가능한지 체크
 - 이것까지 완료되면 최종 예약 확정하여 예약 정보 DB에 등록

- **booking agent의 반환값** → “예약 확정 여부”를 return

- `booking_available` : 예약 가능
 - `booking_rejected` : 예약 불가
 - 대체 시간 제안
 - 결제 대기 필요
 - 사람 검토 필요
- ver1 : 정보를 다 받으면 바로 예약 확정 (ver1에서는 선입금 대기 고려하지 않음)
 - 따라서 기타 기능들은 나중에 구현하고, 일단 여기서는 `booking_available` 과 `booking_rejected` 두 가지 반환값만 있다고 가정하자.